

Cassandra Wide-Column Store

Chapter 06 - Sub-Chapter 03

NoSQL Databases Series

What is Cassandra?	Masterless ring -- built for global scale
Data Model	Keyspace, Table, Partition Key, Clustering
Primary Key Design	The most critical skill in Cassandra
Replication & Consistency	ONE, QUORUM, ALL -- tunable per query
Spring Data Cassandra	Entities, Repositories, CassandraTemplate
When to Use Cassandra	Time-series, high writes, multi-datacenter

1

What is Apache Cassandra?

The wide-column store built for massive scale

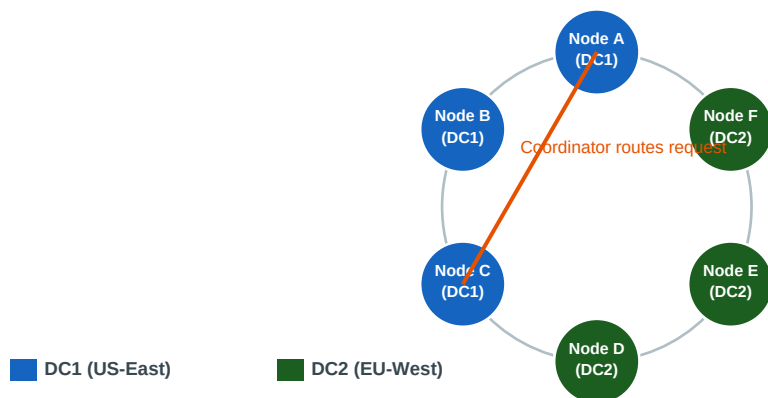
Imagine you are building a system that must track every click, every sensor reading, every order — from millions of users, globally, 24/7 — with no downtime ever. That is the problem Cassandra was designed to solve.

Apache Cassandra was created at Facebook in 2008 to power their inbox search. It combines ideas from two influential papers: Google Bigtable (the wide-column data model) and Amazon Dynamo (the peer-to-peer ring with no single master). Today it is used by Netflix, Apple, Uber, Instagram, and thousands of others.

Key properties that define Cassandra:

- **Masterless ring topology:** Every node is equal — no primary, no secondary. Any node can serve any request.
- **Linear scalability:** Double the nodes → double the throughput. Netflix runs Cassandra at tens of thousands of nodes.
- **Multi-datacenter replication:** Data is replicated across geographies automatically. Survive entire datacenter failures.
- **Tunable consistency:** Choose between speed and safety per query — ONE, QUORUM, ALL.
- **Wide-column model:** Tables with flexible columns, optimized for fast writes and range scans on sorted data.
- **High write throughput:** Writes go to an in-memory structure (Memtable) + commit log. Never a slow disk seek on write.

Cassandra Ring Architecture — No Single Point of Failure



Real-world analogy: Think of Cassandra as a circular ring of warehouses. Each warehouse stores a copy of some goods. When you order something, the nearest warehouse ships it. If one warehouse burns down, others still have the stock. You never wait for a 'head office' to approve the order.

2 Cassandra Data Model

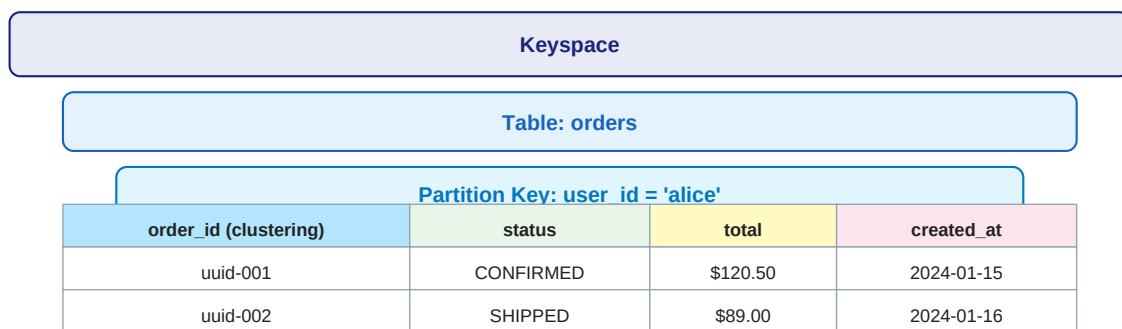
Keyspace → Table → Partition Key → Clustering Columns

Cassandra looks like SQL on the surface — you use CQL (Cassandra Query Language) which resembles SQL. But underneath, the storage model is completely different and this shapes everything about how you design tables.

The hierarchy:

- **Keyspace** — like a database schema. Defines replication strategy.
- **Table** — like an SQL table, but optimized for a specific query pattern.
- **Partition Key** — determines which node stores the data. All rows with the same partition key are stored together on disk.
- **Clustering Columns** — sort order within a partition. Enable efficient range queries.
- **Regular Columns** — the actual data.

Cassandra Data Model — Keyspace → Table → Partition → Row



Rows sorted by clustering column (order_id)

Creating a Keyspace and Table:

```
-- Keyspace: replication across 2 datacenters, 3 copies each
CREATE KEYSPACE ecommerce
WITH REPLICATION = {
  'class': 'NetworkTopologyStrategy',
  'us-east': 3,
  'eu-west': 3
};

USE ecommerce;

-- Table designed for query: "get all orders for a user, sorted newest first"
CREATE TABLE orders_by_user (
  user_id    TEXT,           -- Partition Key: routes to correct node
  order_id   UUID,           -- Clustering Column: sorted within partition
  status     TEXT,
  total      DECIMAL,
  created_at TIMESTAMP,
  PRIMARY KEY (user_id, order_id) -- (partition_key, clustering...)
) WITH CLUSTERING ORDER BY (order_id DESC); -- newest first
```

Why this matters: In Cassandra you design the table around the query, not around the entity. If you need to query orders by user AND by status, you create TWO tables — one per query pattern. This is called

'denormalization by design' and is the biggest mindset shift from SQL.

Inserting and Querying Data (CQL):

```
-- Insert an order
INSERT INTO orders_by_user (user_id, order_id, status, total, created_at)
VALUES ('alice', uuid(), 'CONFIRMED', 120.50, toTimestamp(now()));

-- Query: all orders for alice (uses partition key – fast!)
SELECT * FROM orders_by_user WHERE user_id = 'alice';

-- Query: last 10 orders for alice
SELECT * FROM orders_by_user WHERE user_id = 'alice' LIMIT 10;

-- Query: orders in a date range (clustering column range scan)
SELECT * FROM orders_by_user
  WHERE user_id = 'alice'
     AND order_id > minTimeuuid('2024-01-01 00:00+0000')
     AND order_id < minTimeuuid('2024-02-01 00:00+0000');

-- This would FAIL – you cannot filter on non-key columns without ALLOW FILTERING
-- SELECT * FROM orders_by_user WHERE status = 'CONFIRMED'; -- ERROR
```

The golden rule: Always include the partition key in your WHERE clause. Queries without the partition key require a full table scan across all nodes — never do this in production.

3 Primary Key Design

The most critical skill in Cassandra development

In Cassandra, getting your primary key wrong is catastrophic — it causes hot partitions, slow queries, or data that cannot be retrieved efficiently. Let us walk through a real example.

Scenario: IoT sensor platform

You have 10,000 sensors each sending readings every second. You need to query: 'Give me all readings for sensor X in the last hour, sorted by time.'

Bad design — single partition key on sensor_id:

```
-- BAD: All data for one sensor goes to ONE partition forever
-- A sensor sending 1 reading/sec = 86,400 rows/day = 31M rows/year
-- Cassandra has a soft limit of ~100MB per partition
CREATE TABLE sensor_readings_bad (
    sensor_id TEXT,
    reading_ts TIMESTAMP,
    value      DOUBLE,
    PRIMARY KEY (sensor_id, reading_ts)
);
-- Problem: "hot partition" — one node gets overloaded
-- Problem: partition grows unbounded — performance degrades over time
```

Good design — composite partition key with time bucket:

```
-- GOOD: Partition by sensor + day (bucket)
-- Each partition holds 86,400 rows max — manageable!
CREATE TABLE sensor_readings (
    sensor_id TEXT,
    day       DATE,           -- bucket: partition per day
    reading_ts TIMESTAMP,     -- clustering: sorted within day
    value     DOUBLE,
    unit      TEXT,
    PRIMARY KEY ((sensor_id, day), reading_ts) -- composite partition key
) WITH CLUSTERING ORDER BY (reading_ts DESC);

-- Insert
INSERT INTO sensor_readings (sensor_id, day, reading_ts, value, unit)
VALUES ('sensor-42', '2024-01-15', toTimestamp(now()), 23.7, 'celsius');

-- Query last hour for sensor-42 today (hits exactly one partition)
SELECT * FROM sensor_readings
WHERE sensor_id = 'sensor-42'
    AND day = '2024-01-15'
    AND reading_ts > toTimestamp(now()) - 3600s
LIMIT 100;
```

Key insight: The partition key controls data distribution. The clustering key controls sort order within a partition. Together they answer: 'Where does data live?' and 'In what order is it stored?'

Partition Key Selection Rules:

- High cardinality — many distinct values ensure data spreads across all nodes evenly.
- Frequently used in WHERE clauses — every query must include the partition key.

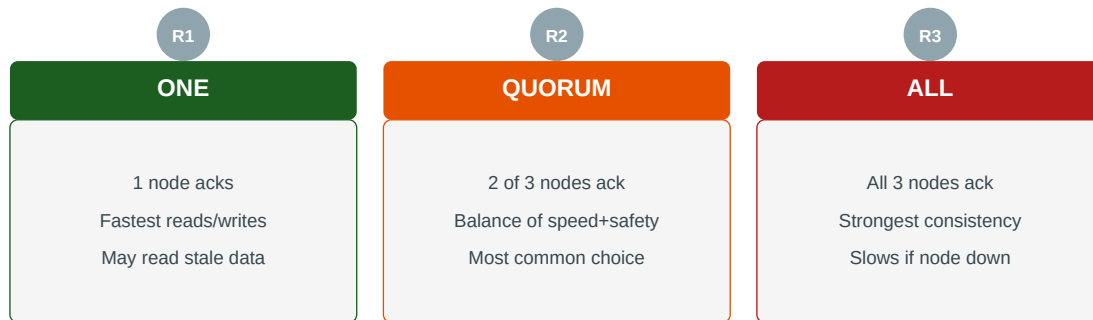
- Bounded size — partition should never exceed ~100MB (Cassandra 4.x relaxes this but principle holds).
- Avoid sequential IDs as sole partition key — time-based UUIDs cause 'hot last node' problem.
- Use composite partition keys (col1, col2) to control bucket size.

4 Replication and Consistency

Tuning the CAP tradeoff per query

Cassandra gives you a dial between consistency and availability. You do not have to pick one globally — you choose per read and per write operation. This is one of Cassandra's most powerful (and initially confusing) features.

Consistency Levels — Replication Factor 3



Replication Factor (RF):

RF=3 means every row is stored on 3 different nodes. If one node goes down, you still have 2 copies. Most production setups use RF=3.

Quorum formula — the sweet spot:

```
-- Quorum = floor(RF/2) + 1
-- RF=3 → Quorum = 2 nodes must agree

-- Write with QUORUM: 2 of 3 replicas must confirm the write
-- Read with QUORUM: 2 of 3 replicas must return data, Cassandra picks latest

-- If Write QUORUM + Read QUORUM → you always read your own writes (strong consistency)
-- Because: write touches 2 nodes, read touches 2 nodes → at least 1 overlap

-- CQL consistency level per statement:
CONSISTENCY QUORUM;
SELECT * FROM orders_by_user WHERE user_id = 'alice';

-- Or use ONE for maximum speed (analytics, non-critical reads)
CONSISTENCY ONE;
SELECT COUNT(*) FROM sensor_readings WHERE sensor_id='s1' AND day='2024-01-15';
```

Hinted Handoff — writes survive node failures:

If a replica node is temporarily down when a write arrives, Cassandra stores a 'hint' on the coordinator. When the downed node comes back online, the hint is replayed. This is why Cassandra achieves 'eventual consistency' — the data gets there eventually.

Read Repair — consistency on reads:

When Cassandra reads with QUORUM, it compares responses from multiple replicas. If they differ (due to missed writes), it repairs the stale replica in the background. This keeps replicas in sync over time without expensive coordination.

5 Spring Data Cassandra

Java integration with annotations and repositories

Spring Data Cassandra wraps the DataStax Java driver and gives you a familiar Spring-style experience: entity classes, repositories, and template methods — the same patterns you know from Spring Data JPA, adapted for Cassandra's model.

Maven dependency:

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra</artifactId>
</dependency>
```

application.yml configuration:

```
# application.yml
spring:
  cassandra:
    contact-points: cassandra-node1.example.com, cassandra-node2.example.com
    port: 9042
    keyspace-name: ecommerce
    local-datacenter: us-east
    username: appuser
    password: secret
    schema-action: CREATE_IF_NOT_EXISTS # auto-create tables in dev
```

Entity class — maps Java to Cassandra table:

```
import org.springframework.data.cassandra.core.cql.PrimaryKeyType;
import org.springframework.data.cassandra.core.mapping.CassandraType;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKeyColumn;
import org.springframework.data.cassandra.core.mapping.Table;
import com.datastax.oss.driver.api.core.type.DataTypes;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

@Table("orders_by_user") // maps to Cassandra table name
public class OrderByUser {

    @PrimaryKeyColumn(name = "user_id", type = PrimaryKeyType.PARTITIONED)
    private String userId; // partition key

    @PrimaryKeyColumn(name = "order_id", ordinal = 0, type = PrimaryKeyType.CLUSTERED)
    @CassandraType(type = DataTypes.TIMEUUID)
    private UUID orderId; // clustering column

    @Column("status")
    private String status;

    @Column("total")
    private BigDecimal total;

    @Column("created_at")
    private Instant createdAt;
```



```

    // getters, setters, constructors
}

```

Repository — zero boilerplate for common queries:

```

import org.springframework.data.cassandra.repository.CassandraRepository;
import org.springframework.data.cassandra.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.UUID;

@Repository
public interface OrderRepository extends CassandraRepository<OrderByUser, UUID> {

    // Spring Data derives CQL from method name
    List<OrderByUser> findByUserId(String userId);

    // Custom CQL for range query with limit
    @Query("SELECT * FROM orders_by_user WHERE user_id = ?0 LIMIT ?1")
    List<OrderByUser> findRecentOrders(String userId, int limit);
}

```

Service layer — using the repository:

```

import org.springframework.stereotype.Service;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;
import java.util.UUID;

@Service
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public OrderByUser placeOrder(String userId, BigDecimal total) {
        OrderByUser order = new OrderByUser();
        order.setUserId(userId);
        order.setOrderId(UUID.randomUUID());
        order.setStatus("PENDING");
        order.setTotal(total);
        order.setCreatedAt(Instant.now());
        return orderRepository.save(order); // written to Cassandra
    }

    public List<OrderByUser> getUserOrders(String userId) {
        return orderRepository.findByUserId(userId); // partition key query
    }

    public List<OrderByUser> getRecentOrders(String userId) {
        return orderRepository.findRecentOrders(userId, 10);
    }
}

```

CassandraTemplate for advanced queries:

```
import org.springframework.data.cassandra.core.CassandraTemplate;
import org.springframework.data.cassandra.core.query.Query;
import org.springframework.data.cassandra.core.query.Criteria;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class SensorService {

    private final CassandraTemplate cassandraTemplate;

    public SensorService(CassandraTemplate cassandraTemplate) {
        this.cassandraTemplate = cassandraTemplate;
    }

    public List<SensorReading> getReadings(String sensorId, String day) {
        Query query = Query.query(
            Criteria.where("sensor_id").is(sensorId),
            Criteria.where("day").is(day)
        ).limit(100);
        return cassandraTemplate.select(query, SensorReading.class);
    }
}
```

6 When to Use Cassandra

Match the tool to the problem

Cassandra excels at:

- ✓ **Time-series data at scale:** IoT sensor readings, application metrics, audit logs, clickstream data.
- ✓ **High write throughput:** Millions of writes per second distributed across nodes.
- ✓ **Global multi-datacenter deployments:** Active-active, survive datacenter failures with zero downtime.
- ✓ **Append-only workloads:** Event logs, activity feeds, chat message history (Netflix uses Cassandra for view history).
- ✓ **Predictable low-latency reads:** When you always query by partition key — single-digit millisecond reads.

Cassandra is a poor fit for:

- ✗ **Ad-hoc queries / analytics:** You cannot filter by arbitrary columns. Every query pattern needs its own table.
- ✗ **Relationships and JOINS:** No foreign keys, no joins. Cassandra is not a relational database.
- ✗ **ACID transactions across rows:** Cassandra has lightweight transactions (LWT) for single partition, but multi-partition ACID is not supported.
- ✗ **Small datasets:** The overhead of Cassandra's distributed architecture is not worth it for datasets that fit in Postgres.
- ✗ **Frequent updates to existing rows:** Cassandra's immutable SSTable model makes heavy update patterns inefficient.

Real-world use cases:

Company	What they store in Cassandra	Scale
Netflix	Viewing history, billing records, A/B test assignments	Petabytes, 2500+ nodes
Uber	Driver/rider location, trip events, surge pricing events	Billions of writes/day
Apple	iCloud metadata, device registrations, push tokens	Hundreds of PB
Instagram	Media metadata, follower graphs (historical)	Billions of rows
Discord	Chat message storage	Trillions of messages

Summary — the one-line rule:

Use Cassandra when you need to write a lot of data fast, read it back by a known key, and need it to keep working even when servers fail. If you need flexible queries or relationships, use Postgres or MongoDB instead.

Key Rules

- Always query by partition key — never rely on ALLOW FILTERING in production.
- Design tables for queries, not for entities — one query pattern = one table.
- Choose RF=3 and QUORUM consistency for most production workloads.
- Use time-bucket composite partition keys for time-series to avoid hot partitions.
- Cassandra is eventually consistent by default — design for it, do not fight it.